Hermes Frontend Design Studio

Product Requirements Document & Functional Specification

A guided visual workspace for turning project context into a considered user experience, a chosen visual direction, and a working frontend.

Document	Version 1.0
Prepared	November 2026
Product type	Hermes plugin concept
Audience	Product owner, designers, frontend engineers, plugin builders, evaluators

Product promise
Help people make frontend decisions with their eyes before they have to judge them in code. Hermes understands the project and its users, presents a small set of reasoned UX and UI directions, then lets the user inspect and refine the chosen design while it becomes real code.

1. Executive summary

The studio addresses a specific pain in AI-assisted frontend work: users can explain a desired experience, but code-only responses make it difficult to judge information hierarchy, navigation, visual style, and interactions. The plugin makes those choices visible and reviewable inside Hermes.

Product definition

A conversational design workflow paired with a visual canvas and preview. It elicits project context, drafts UX alternatives, creates visual UI concepts, supports direct and conversational editing, and applies the approved direction to the user's frontend project.

Primary outcome

The user reaches a working frontend that reflects an explicitly chosen UX and visual direction, with fewer cycles spent searching references, interpreting code, and correcting a mismatched first implementation.

Design principles

• Fit the design to users, tasks, domain, and constraints; do not apply a universal template.

• Show before committing: meaningful design choices are visible before production code changes.

• Keep the user in control through alternatives, approval points, undo, and version comparison.

• Connect preview to implementation: visual edits persist in the rendered app and code.

• Be honest about sources: distinguish generated concepts, linked references, and reusable components.

• Make progress legible: show what Hermes understood, what is uncertain, and what changed.

Product boundaries

This is a frontend design and implementation companion. It is not a general image board, replacement for a full design suite, automatic deployment tool, or guarantee that arbitrary third-party sites can always be searched or copied.

2. Goals, non-goals, and success measures

Goals

• Understand project, users, primary tasks, and constraints before proposing design.

• Offer 2-3 meaningfully different UX directions with trade-offs.

• Offer 2-3 visual UI concepts grounded in chosen UX and style preferences.

• Accept style descriptions, references, and component-library preferences.

• Provide a visual preview where elements can be selected and edited.

• Implement the approved direction and keep visual output connected to code.

• Support refinement, comparison, and recovery from unwanted changes.

Non-goals for initial release

• Replacing professional design suites for detailed vector authoring or team design operations.

• Guaranteeing access to every site, search engine, paid gallery, or private design system.

• Copying protected designs pixel-for-pixel or presenting references as original.

• Building backend services or deploying changes without user action.

• Generating unlimited full-site alternatives before the user chooses a direction.

Success measures (targets to validate)

Measure	Initial target / definition
Time to useful visual	First project-specific concept visible within 5 minutes after required context.
Direction clarity	80% of evaluators can explain differences among offered options.
Edit fidelity	A visual edit appears in preview and survives refresh/rebuild.
User control	Code-writing stages show status and a recoverable prior version.
Reliability	No silent loss of approved edits; failures have recovery paths.
Adoption signal	Users choose a direction and refine it before implementation.

3. Users and use cases

User	Need	Typical use
Solo builder / founder	Turn a product idea into credible frontend.	Describe app, compare, select, refine, build.
Frontend engineer	Get a visual target compatible with an existing codebase.	Provide stack constraints; inspect and adjust.
Product designer	Explore alternatives while retaining UX rationale.	Review flows, set style, edit, hand off.
Agency / consultant	Create explainable proposals and gather client feedback.	Present options, preserve decisions and versions.
Student / prototyper	Learn how users and tasks shape interface choices.	Compare alternatives and iterate.

Core jobs to be done

- When I describe a product, help me decide what users need and how the interface supports it.
- When I cannot picture a code change, show a visual option I can react to.
- When I have references, translate qualities I like into an original, appropriate design.
- When I select or describe a change, update the visible design and frontend consistently.
- When an edit goes wrong, let me compare, undo, or restore a prior state.

4. Project intake and design brief

Field	Examples / behavior
Product context	What it does, domain, maturity, docs or product URL.
Audience	Roles, skill, accessibility needs, devices, locale.
Primary tasks	Top user goals, frequency, urgency, success criteria.
Pages and content	Requested routes, page purpose, key data and relationships.
UX preferences	Navigation, onboarding, density, workflows, empty/error states.
Visual preferences	Mood words, styles, references, brand colors, typography, avoid-list.
Implementation constraints	Existing project, framework, libraries, tokens, responsive targets.
Practical constraints	Deadline, content readiness, localization, performance.

Intake behavior

• Accept free-form context first; do not force a long form.

• Extract a draft brief and show it in editable plain language.

• Ask focused high-impact questions; bundle related questions and offer defaults.

• Allow “make reasonable assumptions” and display assumptions for review.

• Do not treat page count or style label as sufficient UX evidence.

• Allow users to add, remove, rename, and prioritize pages.

Readiness gate

Before proposing directions, identify at least a product purpose and one primary audience/task. If missing, ask briefly. If the user declines, proceed with clearly marked assumptions and lower-confidence recommendations.

5. Functional requirements

ID / capability	Requirement
FR-01 Brief extraction	Summarize product, audience, tasks, pages, constraints, uncertainties; user edits every field.
FR-02 UX alternatives	Generate 2-3 distinct flows/navigation concepts. Each includes user/task fit, page structure, interaction, strengths, trade-offs, assumptions.
FR-03 UX review	Select, revise, combine compatible parts, or request recommendation with rationale.
FR-04 Reference discovery	Accept URLs, images, style terms, source names. Search only accessible sources; show link and observed qualities.
FR-05 Reference integrity	Separate references from generated concepts; disclose access failures and attribute sources; avoid wholesale copying.
FR-06 UI concepts	Generate 2-3 visual concepts appropriate to project and selected UX.
FR-07 Concept comparison	Show comparable screens with typography, palette, density, components, responsive behavior.
FR-08 Style input	Accept natural language, styles, brand inputs, library preference; translate labels into concrete properties.
FR-09 Preview	Navigable routes, viewport sizes, loading/empty/error states, visible concept/version identity.
FR-10 Element selection	Selection identifies element and exposes appropriate properties/context.
FR-11 Visual edits	Text, color, typography, spacing, size, alignment, border, radius, layout, icon, image, interaction, animation.
FR-12 Conversational edits	Change selected element/page/design; clarify ambiguous target or effect.
FR-13 Components	Suggest compatible library elements; show source and dependency implications.
FR-14 Responsive behavior	Preview desktop/tablet/mobile; preserve usability and hierarchy.
FR-15 Accessibility	Flag contrast, focus, labels, keyboard, motion, semantic issues; offer fixes.
FR-16 Implementation	After approval, change project code and show affected areas and build status.
FR-17 Design-code sync	Direct/chat edits update preview and implementation; label mock-only work clearly.
FR-18 Versioning	Checkpoint brief, UX/UI choices, significant edits; compare and restore.
FR-19 Approval	Confirm before replacing substantial work, adding dependencies, deleting user content, or broad edits.
FR-20 Handoff	Export rationale, tokens, pages, components, references, open decisions, implementation status.

6. User journeys and scenarios

A. New frontend from context

• User shares project context and requests frontend design.

• Studio extracts users, tasks, pages, constraints; user edits brief.

• Hermes offers 2-3 UX directions with page map and interaction flows.

• User chooses, combines, or revises; assumptions and decisions are saved.

• User enters a style, gives references/library preference, or requests original concepts.

• Studio shows 2-3 UI concepts based on selected UX.

• User selects a concept and opens the live preview.

• User edits directly and/or through chat; changes preview and checkpoint.

• User approves implementation; Hermes updates code and runs preview/build.

• User reviews viewports, requests changes, or restores a checkpoint.

B. Existing codebase redesign

• User selects project and requests redesign.

• Studio summarizes routes/components and asks what may change.

• It preserves constraints and flags uncertainty.

• User reviews directions and approves scope before edits.

• Compare preview to current experience; implement only after approval.

C. Inspiration-led exploration

• User shares URL/image or describes a mood.

• Studio reports what it could access and summarizes qualities, not a clone.

• User confirms desired qualities; studio generates original concepts.

• If access is blocked, upload an image or continue from description.

D. Targeted visual correction

• User selects a button/card and asks to improve mobile scanning.

• Studio identifies target and proposes visible responsive changes.

• User previews, accepts, adjusts, or undoes.

• Change updates code and remains after reload.

Example

User: “I’m building a meal-planning app for busy parents: weekly plan, recipes, shopping list, dietary preferences.” **Studio:** Summarizes audience and tasks, flags assumptions, asks one focused question. It offers week-first, recipe-first, and task-first UX approaches. User chooses week-first, requests warm minimal styling and a high-contrast shopping mode. Studio shows three comparable concepts. User selects one, changes the primary action color, and asks for more mobile whitespace. The edit is previewed and checkpointed. After approval, Hermes updates the frontend and shows mobile/desktop previews and a change summary.

7. Business logic and rules

Rule	Behavior
BR-01 Intent first	User context and constraints outweigh generic trends.
BR-02 UX before UI	Resolve core flows and hierarchy before treating styling as the answer.
BR-03 Distinct options	Alternatives differ in meaningful decisions, not superficial color changes.
BR-04 Explain recommendation	Connect rationale to users, tasks, constraints, and trade-offs.
BR-05 Limit overload	Default to 2-3 options; keep compared screens consistent.
BR-06 Preserve choices	Approved constraints persist until user changes them.
BR-07 Scope edits	Selected-element edits do not silently affect unrelated pages.
BR-08 Shared components	Warn when a shared component change affects multiple pages.
BR-09 Destructive approval	Confirm deletion, broad replacement, dependency addition, or rewrite.
BR-10 Checkpoint	Save recoverable state before significant code changes.
BR-11 Source transparency	Attribute references; label generated work and access status.
BR-12 No exact-copy promise	Use high-level qualities; do not claim copied work is original.
BR-13 Honest status	Distinguish mock, partial interaction, build-ready, and unavailable.
BR-14 No silent fallback	Explain search/preview/build failure and next steps.
BR-15 Accessibility	Honor reduced motion and user accessibility needs.
BR-16 Consistency	Use tokens/components where appropriate; disclose intentional overrides.

8. Interface layout, controls, and states

Region	Content / interaction
Conversation	Context, explanations, questions, decisions, edit requests.
Design canvas	UX map, alternatives, selected page, or live preview.
Properties panel	Selected element details and contextual controls.
Project/version strip	Project, version, viewport, route, save/build status, history.
Reference shelf	Examples, source links, access results, style notes.
Decision bar	Choose direction, request changes, approve, undo, compare.

Required states

State	User sees / can do
Idle / no project	Start design, open project, enter context.
Gathering brief	Editable extraction, questions, assumptions.
Generating UX	Progress, cancel; brief preserved.
UX review	Compare, inspect flows, choose or revise.
Searching references	Source/status/progress; cancel or continue.
Generating UI	Progress and selected UX/style context.
UI review	Comparable concepts; select or request revision.
Preview ready	Navigate, select, resize viewport, edit, undo.
Applying edit	Target and pending change; cancel when safe.
Implementation approval	Scope, affected areas, dependency changes.
Building	Progress and last successful checkpoint.
Build success	Preview, change summary, continue or finish.
Build failure	Readable issue and retry/rollback choices.
Search unavailable	Explain limitation; accept upload/description.
Unsaved / stale	Offer save, discard, compare.
Conflict	Preserve user changes and offer review choices.

Interaction behavior

- Selection outline identifies element; Escape clears it.
- Property controls use plain terms and live preview; advanced values remain available.
- Chat requests use current selection when clear; otherwise ask target/scope.
- Inspect hover, focus, pressed, disabled states for interactive components.
- Viewport change does not discard edits.
- Keyboard users can reach all controls; provide non-drag alternatives.

9. Inputs and outputs

Input	Forms	Output
Project context	Chat, docs/URLs, codebase	Editable brief, assumptions, questions.
Audience/tasks	Natural language or lists	User/task profiles, flows, page needs.
Style direction	Text, style name, brand guide/tokens	Interpreted constraints and visual direction.
References	URL, image, search	Attributed cards, access state, extracted qualities.
Component preference	Library, project dependencies	Compatible recommendations and implications.
Visual selection	Pointer, keyboard, element tree	Target context and editable properties.
Edit/approval	Chat, controls, confirm/decline	Previewed change, scope, checkpoint.

Expected outputs

• Editable brief and assumptions.

• UX direction cards with page maps, flows, rationale, trade-offs.

• UI concepts with representative screens, tokens, responsive samples.

• Attributed references and access notes.

• Navigable preview with direct element editing.

• Working frontend after approval.

• Change summary, build status, version history.

• Exportable handoff summary.

10. Edge cases, errors, and recovery

Case	Expected handling
Insufficient context	Ask audience/task; allow explicit assumptions.
Conflicting requirements	Name conflict and consequence; ask priority.
Too many pages	Group core/later pages; confirm scope.
Vague style	Translate to concrete traits and verify with visual preview.
Reference blocked/paywalled	Disclose; offer image upload or description.
Unclear reference image	State confidence; ask what qualities matter.
Exact clone requested	Offer high-level qualities and original design.
Unsupported component	Explain; offer compatible alternative.
No runnable preview	Show issue or static preview labeled as such.
Build failure	Preserve last good state; explain and retry/rollback.
User edits code concurrently	Detect conflict; preserve edits and offer merge/review.
Element unselectable	Offer accessible element tree/search.
Shared component selected	Warn of broad impact; local override or shared update.
Animation harms accessibility	Flag and offer restrained/reduced-motion behavior.
Broad redesign mid-edit	Checkpoint pending work; clarify keep/discard.
Connection lost/cancelled	Retain brief, concepts, checkpoints; resume/restart.
Dependency needed	Explain package/reason; request approval and offer fallback.
Preview/code mismatch	Mark inconsistent; rebuild from approved checkpoint.
No useful concept	Do not fake success; revise brief or offer lower-confidence draft.

11. Quality, accessibility, and trust

• Responsive layouts remain usable on phone, tablet, desktop; surface overflow issues.

• Accessibility includes semantic structure, keyboard use, focus, contrast, labels, reduced motion, clear errors.

• Preserve inputs and last successful outputs through cancellation and failures.

• Identify generated concepts, sources, assumptions, limitations, and unverified implementation.

• Explain what project files/reference data are read and whether external services are involved; keep secrets out of previews/logs.

• Show incremental progress and allow safe cancellation.

• Users understand status and next step without frontend jargon.

• Maintain coherent tokens/components and explain intentional overrides.

12. MVP and later releases

MVP: prove core value	Later expansion
Editable brief and assumptions	Team review and shared decision history
2-3 UX directions with maps/trade-offs	Research and analytics-informed recommendations
2-3 UI concepts for representative screens	More reference integrations
Style input and user-provided references	Saved brand kits and org design systems
Interactive preview and basic visual edits	Advanced motion tooling
Chat edits for selected element/page	Fine-grained responsive breakpoint editor
Approval-gated code and preview	Handoff integrations
Undo/checkpoints and build recovery	Visual regression/test generation
Basic accessibility/responsive checks	Team governance and richer audits

MVP cut line

Demonstrate one end-to-end loop: brief -> UX alternatives -> selected UX -> UI concepts -> visual edit -> approved code change -> working preview. Searching many inspiration sites is not required; user-provided references and a small set of accessible sources are enough to prove the core value.

13. Acceptance criteria

• With audience and tasks present, studio returns editable summary and assumptions.

• Ready brief yields 2-3 UX directions that differ in flow/hierarchy; each has pages, a key flow, rationale, trade-offs.

• User can revise/select UX before visual generation.

• Selected UX and style yield 2-3 comparable visual concepts.

• User can identify selected concept and its inputs/references.

• User can select a visible element and change at least text, color, spacing, typography.

• Chat edit updates preview without unintended changes to unrelated areas.

• User can undo and restore checkpoints.

• Before implementation, scope is shown and material existing-project changes require approval.

• Approved implementation appears in code and refreshed preview, or failure has recovery choices.

• Desktop/mobile layouts are inspectable; accessibility limitations are clear.

• Reference access failures are disclosed.

• Destructive actions and dependency additions require explicit approval.

14. Risks and mitigations

Risk	Mitigation
Looks like prompt wrapper	Make visual comparison/editing, approval, and code sync central.
Generic concepts	Ground choices in user/task evidence and visible differentiators.
Mock differs from implementation	Make sync core; label mock-only states and verify after build.
Unreliable references	Support URLs, uploads, descriptive fallback, source status.
Too much process	Progressive disclosure, defaults, early sketches when suitable.
Choice overload	Limit choices and use consistent comparison criteria.
Edits break project	Checkpoint, scope, explain shared effects, rollback.
Brittle library output	Follow existing conventions; disclose additions.
Inappropriate copying	Extract qualities, attribute sources, create original assets/layout.
Accessibility superficial	Include checks throughout concept, preview, and edit workflows.

15. Positioning and distinction

Positioning statement

For people building web products with Hermes who need to judge frontend work visually, Hermes Frontend Design Studio turns project context into user-specific UX options and editable UI concepts, then carries the approved direction into working code. Unlike a code-only assistant or static preview, it makes design choices visible, comparable, and directly revisable before and during implementation.

What it is not

Not a generic “prompt to landing page” generator. Its defining behavior connects user needs to UX, UX to visual concepts, and visual edits to working frontend code.

Competition-ready demonstration

Use one realistic project. Show Hermes extracting audience/tasks, generating contrasting UX directions, producing project-appropriate UI concepts, accepting a direct visual edit, and reflecting it in a working responsive page. Keep the edit loop visible and briefly explain the selected concept’s trade-offs.

16. Glossary

Term	Meaning
UX direction	Structure and interaction approach: tasks, navigation, hierarchy, flows.
UI concept	Visual treatment: type, palette, spacing, components, imagery, states.
Reference	Example used to discuss qualities, with source/access status.
Design token	Reusable value such as color, spacing, type size, radius, shadow.
Checkpoint	Recoverable saved state around meaningful design/code changes.
Working preview	Rendered implementation, distinct from static image/mock.
Shared component	Element used in multiple places; edits may affect several pages.

17. Open decisions for product owner

- Should first demo optimize for new apps, existing codebase redesign, or both?
- Which reference sources are essential, and what happens when they are blocked?
- Which frontend frameworks and component libraries are essential for target users?
- Should concepts cover whole pages or representative screens and flows?
- How much does plugin edit directly versus propose for Hermes to implement after approval?
- What project-file scope and privacy notice should precede analysis?
- Which accessibility checks can be supported in the initial release?
- What qualifies as successful code synchronization in the target Hermes experience?

End of specification. This document defines intended product behavior and user-facing requirements; it does not prescribe implementation architecture.